Social Media App (MERN)

Project Documentation

1. Project Description

To provide a seamless, real-time social platform where users can share their lives through posts and stories while maintaining instant connection through live chat.

2. Scenario

The platform allows to chat, post content and stories, and explore blogs worldwide.

3. System Requirements

I. Software Requirements:

Development Side

- **Operating System:** Windows 10+, macOS (High Sierra+), or Linux (Ubuntu 20.04+).
- **Runtime Environment: Node.js** (LTS version recommended, v18.x or v20.x).
- **Package Manager: npm** (included with Node) or **Yarn**.
- **Database: MongoDB** (Local Community Server or MongoDB Atlas cloud).
- **Version Control: Git** (for code management and GitHub/GitLab deployment).
- **Tools: VS Code** (Editor), **Postman** (API testing), and **Chrome DevTools**.

Technology Stack (Dependencies)

- **Frontend:** React.js, Bootstrap 5, Axios (for API calls), Socket.io-client.
- **Backend:** Express.js, Node.js, Mongoose (ODM), Socket.io, JWT (Authentication), Multer (for media uploads).

II. Hardware Requirements:

Developer Machine (Minimum)

- **Processor:** Dual-core 2.4GHz (Intel i3/i5 or equivalent).

- **RAM: 8GB minimum** (16GB recommended to run VS Code, Chrome tabs, and the Node server simultaneously).
- **Storage:** 20GB of available SSD space for project files and local database storage.

Server Side (Deployment)

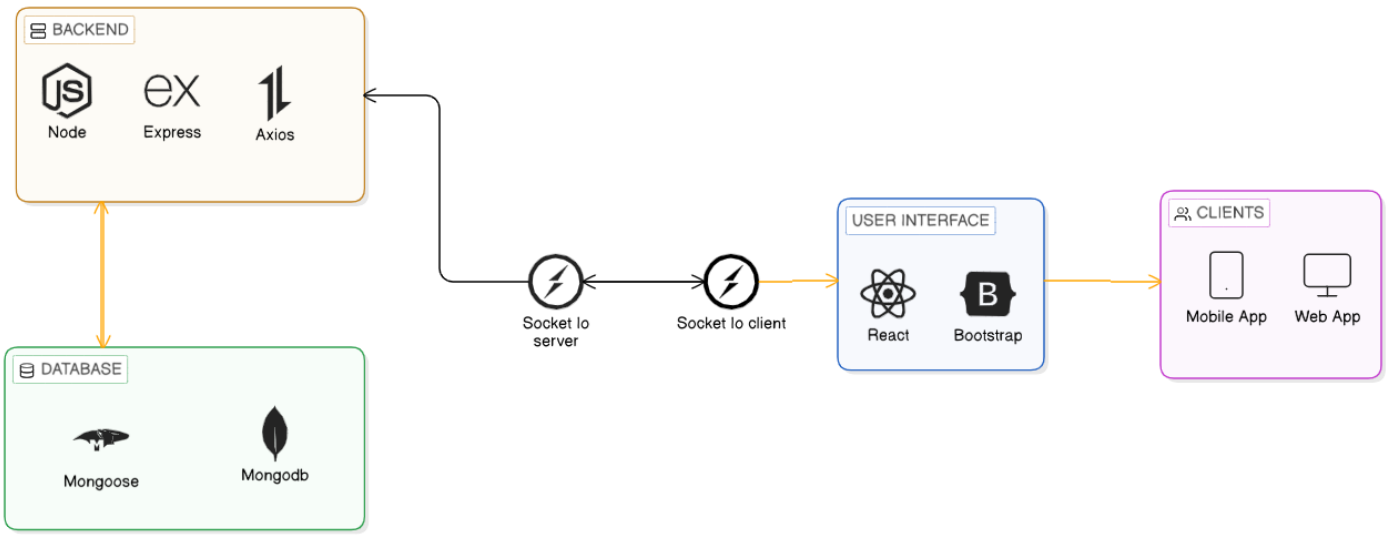
- **Cloud Provider:** AWS, Heroku, Render, or DigitalOcean.
- **Instance Type:** 1 vCPU and 1GB RAM (standard for small-scale MERN apps).

Client Side (End-User)

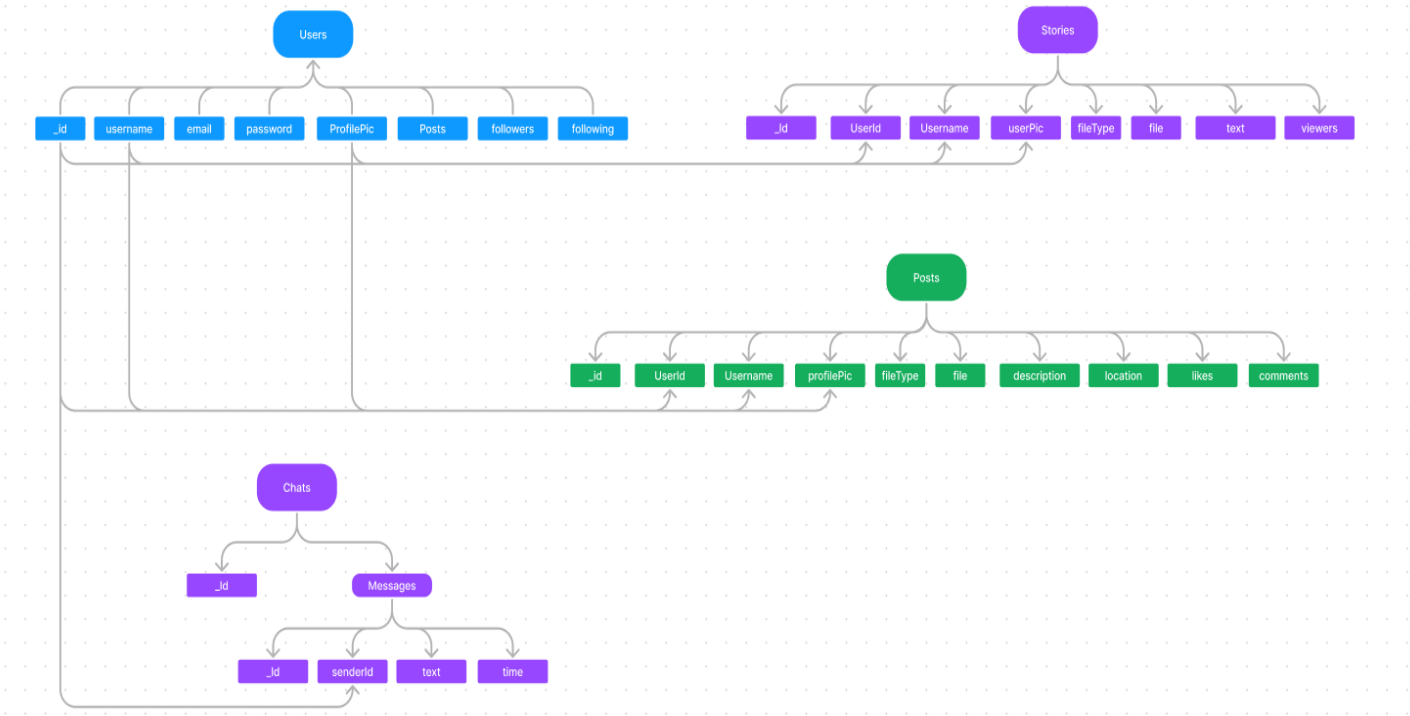
- **Browser:** Modern web browser (Chrome 90+, Safari 14+, Firefox, or Edge).
- **Mobile:** Android 8.0+ or iOS 12+ for the best responsive experience.
- **Network:** Minimum 3G connection; 4G/5G/Wi-Fi recommended for real-time chat and video playback.

4. Project Architecture

- **Technical Architecture**
  Frontend: ReactJS + Bootstrap with Socket.IO client
  Backend: Node.js/Express.js with Socket.IO server
  Database: MongoDB with Mongoose



- **ER Diagram**



- **Features**
  - Update Profile Details & Picture
  - Follow other users
  - Like and Comment on posts
  - Create post and upload with media
  - Create daily stories
  - Personal chats with friends
  - Manage posts (deletion if needed)
- **Roles & Responsibilities**

Development Team (Technical Roles)

- **Frontend Developer (React + Bootstrap)**
  - **Responsibilities:** Developing responsive and interactive user interfaces using **React**. Translating UI/UX wireframes into functional code and ensuring cross-browser compatibility. Managing application state efficiently (often using Redux or Context API).
- **Backend Developer (Node.js + Express.js)**

    - **Responsibilities:** Creating and managing server-side logic and **RESTful APIs** to handle requests between the client and server. Implementing secure user authentication and authorization using **JWT** (JSON Web Tokens).

- **Real-time Specialist (Socket.io)**
    - **Responsibilities:** Implementing bidirectional, real-time communication for features like instant messaging, typing indicators, and live notifications. Managing active socket connections and chat "rooms" for private and group messaging.

- **Database Administrator (MongoDB + Mongoose)**
    - **Responsibilities:** Designing and implementing scalable NoSQL database schemas for users, posts, and chats. Handling data modelling and ensuring efficient data retrieval and storage processes.

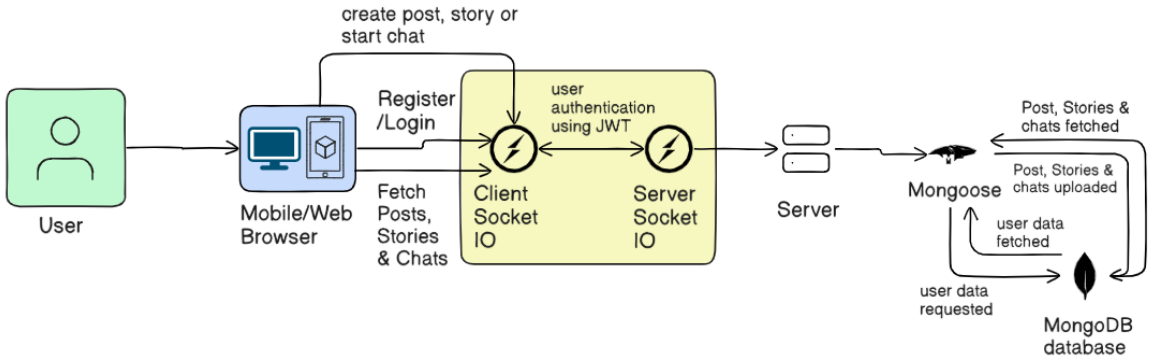
Product & Quality Team

- **Project Manager / Scrum Master**
    - **Responsibilities:** Overseeing the software development lifecycle and ensuring project milestones are met. Facilitating daily stand-ups, managing the product backlog, and coordinating between cross-functional teams.

- **UI/UX Designer**
    - **Responsibilities:** Conducting user research to understand pain points and creating mock-ups that prioritize a seamless user experience. Defining the visual aesthetic, including theme customisation options and responsive layouts for various devices.

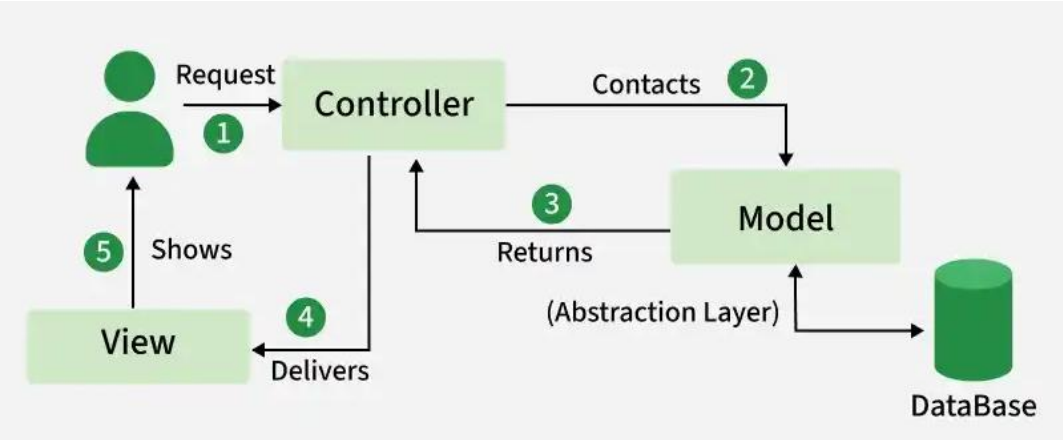
- **QA Engineer / Tester**
    - **Responsibilities:** Identifying and reporting bugs within the application stack through manual and automated testing. Performing end-to-end testing to ensure features like post-creation and real-time chat function correctly under load.

- **User Flow**



a. **Post Creation –**
   Click create post icon > fill the required fields > upload the file > click on Upload button > post uploaded & saved in database > post visible in feed
b. **Story Creation –**
   Click create story icon > fill the required fields > upload the file > click on Upload button > story uploaded & saved in database > story visible in story section
c. **Chatting –**
   Go to chat page > click on user profile to chat > write message > attach media (if needed) > click on send button > message sent and chats saved in database

- **MVC Pattern**



- **Model:** Manages application data and business logic. [Mongoose connects with MongoDB]
- **View:** Handles the user interface and presentation of data. [Route]
- **Controller:** Processes user input and coordinates between Model and View.
- Promotes modularity, reusability, and easier maintenance of the codebase.

5. Project Setup & Configurations

Folder setup:

Firstly, create the folders for frontend and backend to write the respective code and install the essential libraries.

- Client folders.
- Server folders

Installation of required tools:

Open the frontend folder to install necessary tools. For frontend (client), we use:

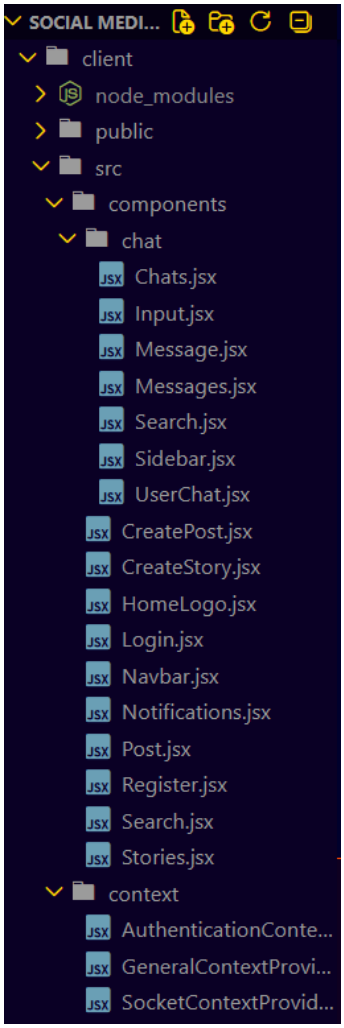
Tools/libraries	Installation Commands
React	npx create-react-app .
Axios	npm install axios
Bootstrap	npm install bootstrap
Socket.io-client	npm install socket.io-client
uuid	npm install uuid

Open the backend folder to install necessary tools. For backend (server), we use:

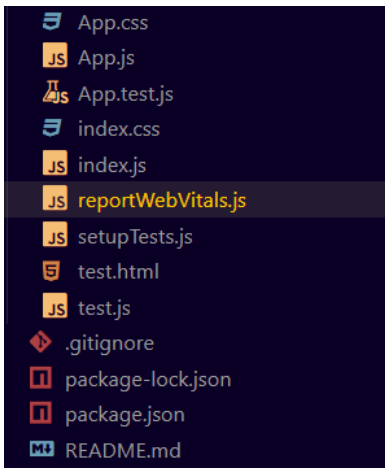
Tools/Libraries	Installation Commands
Express Js	npm install express
Mongoose	npm install mongoose
Bcrypt	npm install bcrypt
Body-parser	npm install body-parser
Cors	npm install cors
Dotenv	npm install dotenv
Http	npm install http
Socket.io	npm install socket.io
Jsonwebtoken	npm install jsonwebtoken
Multer	npm install multer

6. Folder Structure

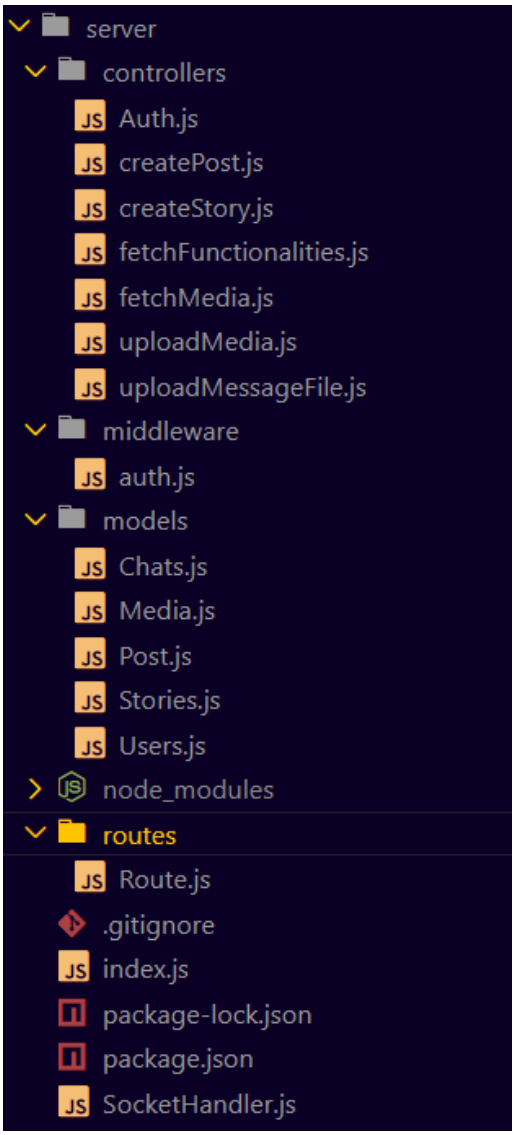
a. **Client:**


SOCIAL MEDI...
client
node_modules
public
src
components
chat
Chats.jsx
Input.jsx
Message.jsx
Messages.jsx
Search.jsx
Sidebar.jsx
UserChat.jsx
CreatePost.jsx
CreateStory.jsx
HomeLogo.jsx
Login.jsx
Navbar.jsx
Notifications.jsx
Post.jsx
Register.jsx
Search.jsx
Stories.jsx
context
AuthenticationConte...
GeneralContextProvi...
SocketContextProvid...

images
about-1.jpg
about-2.jpg
landing-bg.jpg
landing-pic.jpg
ProConectLogo.png
ProConnectLogo_Ho...
ProConnectLogo_Ver...
pages
Chat.jsx
Home.jsx
LandingPage.jsx
Profile.jsx
RouteProtectors
AuthProtector.jsx
LoginProtector.jsx
styles
Chat.css
CreatePosts.css
Home.css
HomeLogo.css
landingpage.css
Navbar.css
Notifications.css
Posts.css
ProfilePage.css
SearchContainer.css
Stories.css



b. **Server:**



7. Running the Application

- Provide commands to start the frontend and backend servers locally.

- **Frontend:** `npm start` in the client directory. [client on http://localhost:3000]
- **Backend:** `node index.js` in the server directory. [server on http://localhost:6001]

8. API Documentation

Method	Endpoint	Description
POST	**/register**	**Register a new user**
POST	**/login**	**Login a existing user**
GET	**/fetchMedia/:fileId**	**Fetch a media file with specified file id**
GET	**/fetchAllPosts**	**Fetch all posts**
GET	**/fetchAllStories**	**Fetch all stories**
GET	**/fetchUserName**	**Fetch username**
GET	**/fetchUserImg**	**Fetches user image**
POST	**/createPost**	**To create a post**
POST	**/createStory**	**To create a story**

9. Authentication

- **Method: JWT (JSON Web Tokens)**.
- **Process:** On login, the server generates a token sent to the client. This token is stored in local Storage or cookies and sent in the **Authorization Header** for protected routes.

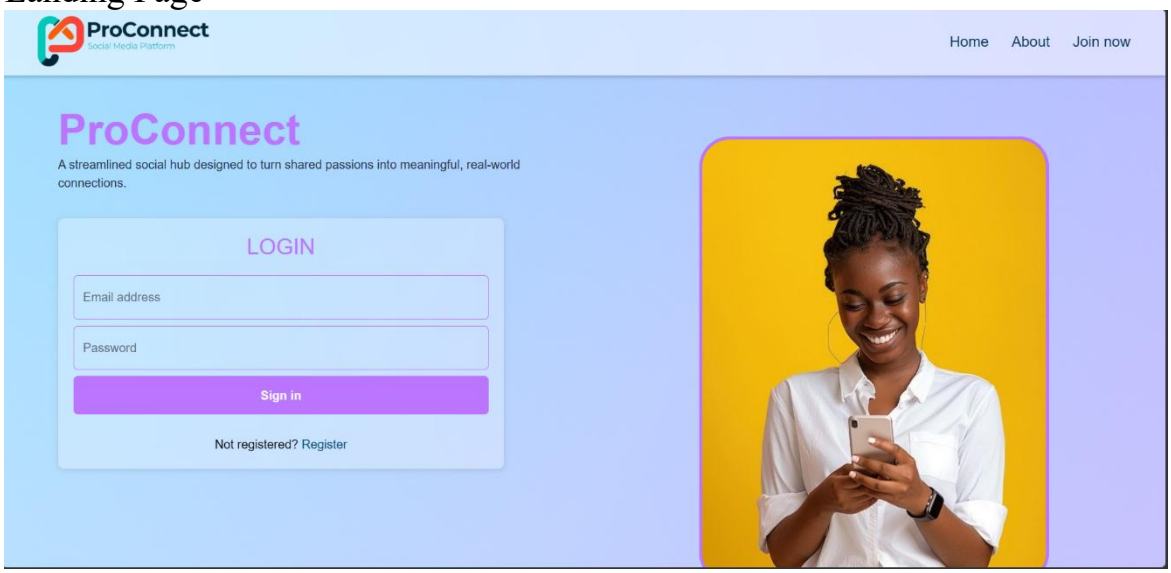
10. Testing

- **Strategy**: Manual UI testing and API testing using Postman
- **Tools:** Postman for backend validation; Chrome DevTools for frontend responsiveness.
- **User Acceptance testing**

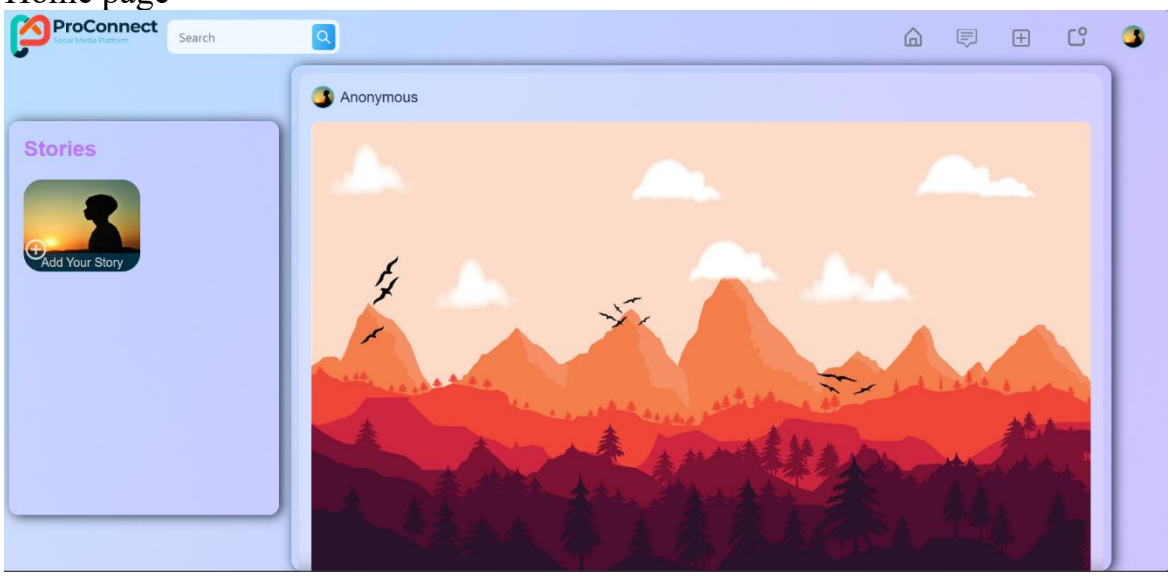
11. Screenshots or Demo

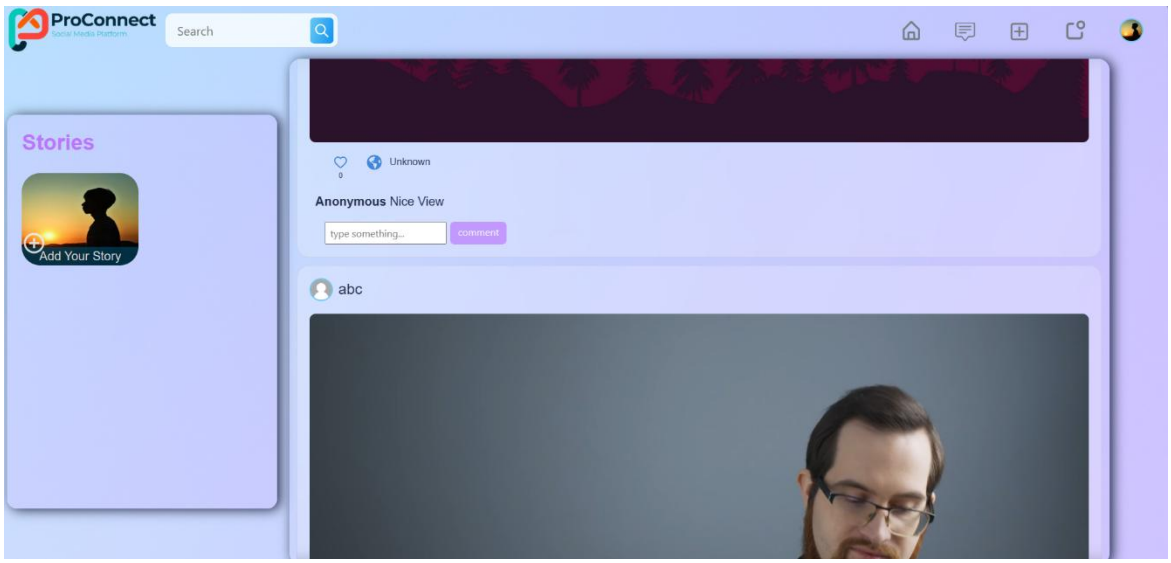
Demo video Link

1. Landing Page

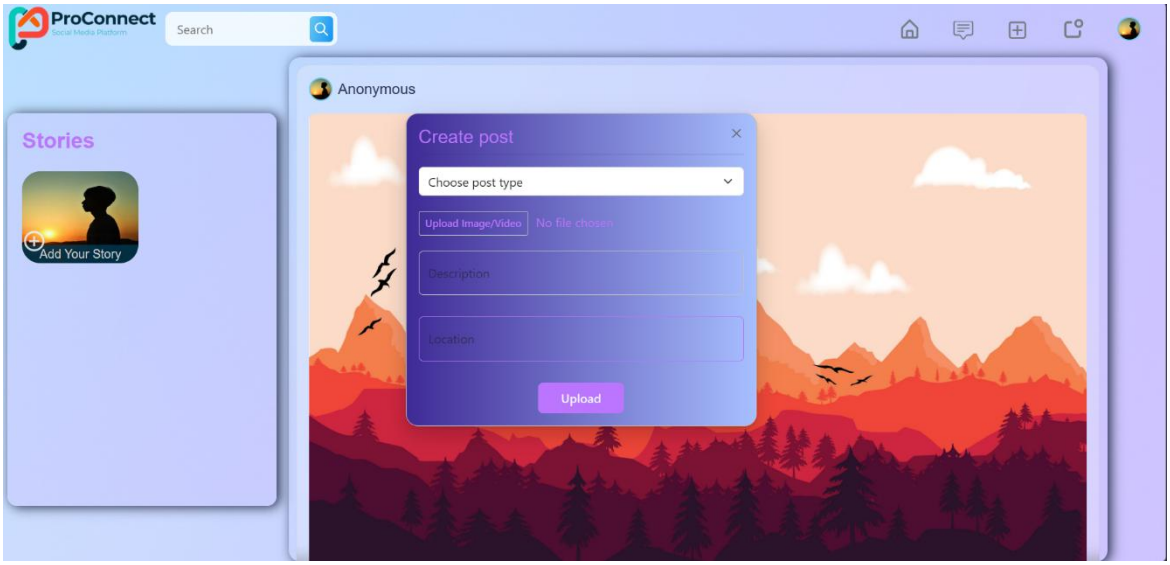


2. Home page

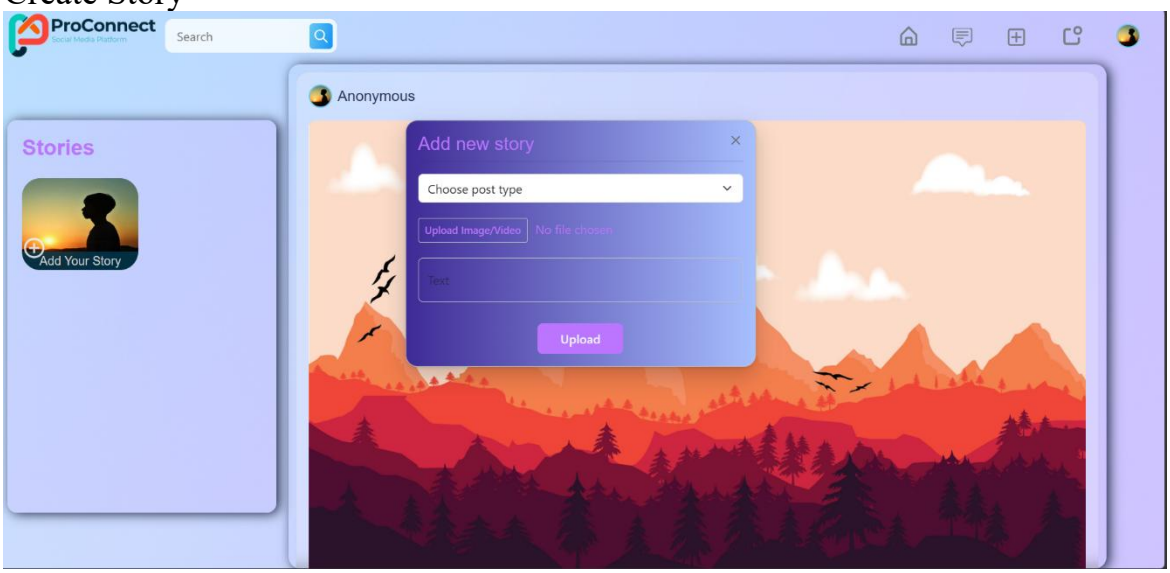




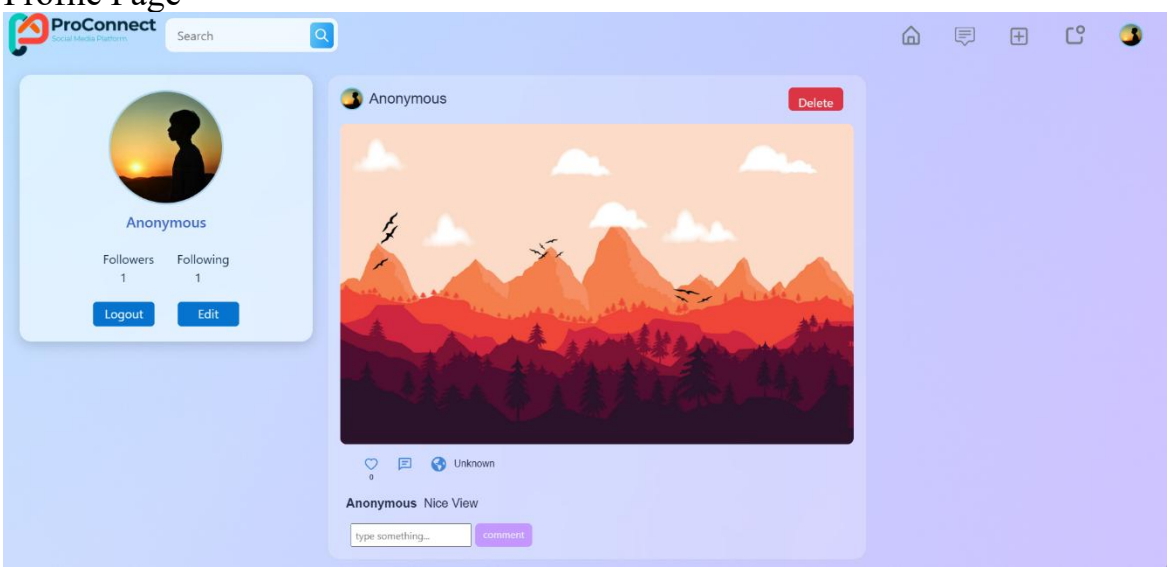
3. Create Post



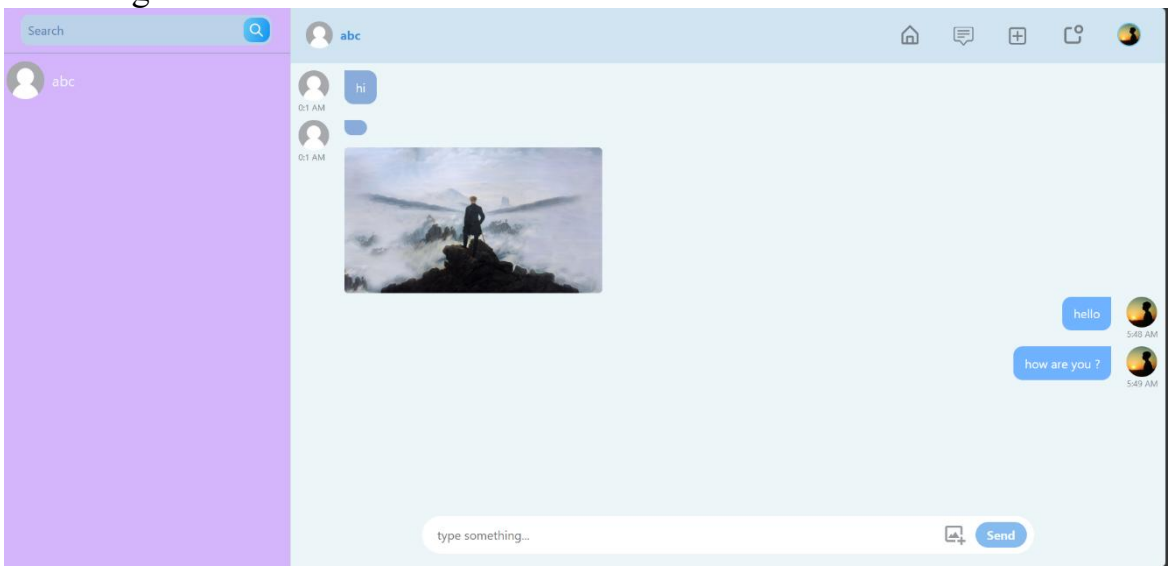
4. Create Story



5. Profile Page



6. Chats Page



12. Known Issues

- High-resolution video uploads may experience slight delays depending on bandwidth.
- Socket connection might drop momentarily during network handovers (e.g., switching from Wi-Fi to Data).

13. Future Enhancements

- Implementing **End-to-End Encryption** for chats.
- Adding **Video Calling** functionality using WebRTC.
- Integrating an **AI-based content moderation** system for posts.